Infosys Springboard Virtual Internship 6.0 Completion Report

Team Details <Do not mention any personally identifiable information like email ID, institute details, mobile phone number etc.>

Batch Number: 13 Receipt and Invoice Digitizer

Start date: 02-02-2026

Names:

Atul Chandra

Shaik Mastan

Thrishala Koduru

Sriya Muthukumar

Rithwik Pachipala

Shailee Choukesy

Internship Duration: 40 days (12 Weeks)

1. Project Title

Receipt and Invoice Digitizer

2. Project Objective

The Receipt and Invoice Digitizer is an AI-driven tool designed to help individuals and businesses eliminate the burden of manual receipt management. By leveraging OCR (PaddleOCR) and a local Large Language Model (Mistral via Ollama), the system automatically extracts key data fields from uploaded receipt and invoice images or PDFs — including Vendor Name, Invoice Number, Date, Total Amount, and Tax. All extracted data is stored per-user in a structured SQLite database, and presented through an interactive analytics dashboard with spending charts and summary statistics. An AI chatbot allows users to query their own expense data in natural language, while features such as dark mode, multilingual support (6 languages), admin dashboard, and secure authentication make the platform accessible and robust for all users.

3. Project description in detail

The Receipt and Invoice Digitizer is a full-stack web application built with Python (Flask) on the backend and HTML/CSS/JavaScript on the frontend. It addresses a common pain point for individuals and small businesses: the manual, error-prone process of tracking physical and digital receipts.

When a user uploads a receipt image or PDF, the system first processes it through PaddleOCR to extract raw text. This text is then passed to the Mistral LLM (running locally via Ollama) which intelligently identifies and extracts structured fields such as the vendor name, invoice number, invoice date, total amount, and tax. A regex-based fallback layer ensures reliable extraction for common formats even when the LLM output is ambiguous. Extracted data is saved to a per-user SQLite database.

The user-facing dashboard displays all stored receipts in a sortable table and provides an analytics section powered by Chart.js, showing daily spending trends and four summary statistics: Total Spent, Total Tax, Receipt Count, and Average per Receipt. Users can view or delete any receipt, and the original file remains accessible via a secure endpoint.

A floating AI chatbot widget — powered by Mistral — allows users to ask natural language questions about their expenses, such as 'How much did I spend this month?' or 'Who is my top vendor?'. The chatbot uses intent detection (both pattern-matching and LLM-based) to route structured queries directly to the database, and falls back to the full Mistral model for open-ended questions.

Additional features include: full dark/light mode toggle using CSS variables, multilingual support via i18next (English, Hindi, Telugu, Tamil, Kannada, and Malayalam), an admin dashboard showing platform-wide usage statistics and per-user receipt history, Google OAuth login, forgot/reset password flow via Flask-Mail, and a responsive design that works across devices.

4. Timeline Overview

Week	Activities Planned	Activities Completed

Week 1	Project scope definition, team role assignment, and requirement gathering	Requirements finalized, team roles assigned (Frontend, Backend, OCR, DB, Auth, Docs)
Week 2	Research on OCR tools, LLM options, and tech stack selection	PaddleOCR and Mistral (via Ollama) selected; Flask + SQLite tech stack finalized
Week 3	Database design and user authentication setup	SQLite schema created for users and receipts; bcrypt authentication implemented
Week 4	OCR pipeline development and PDF support	PaddleOCR integrated; PDF-to-image conversion via PyMuPDF working
Week 5	Mistral LLM integration for field extraction	Mistral extracting Vendor, Invoice #, Date, Total Amount and Tax from OCR text
Week 6	Frontend development: landing page, dashboard, bill digitizer	Landing page, login, registration, and dashboard UI completed with pink/white theme
Week 7	Admin dashboard, chatbot, analytics, multilingual and dark mode	Admin panel, AI chatbot widget, Chart.js analytics, i18next multilingual, dark mode all completed
Week 8	Testing, debugging, documentation, and final deployment	All features tested and debugged; GitHub repository updated; documentation completed

5a. Key Milestones

Milestone	**Description**	**Date**
Project Kickoff	Project objectives, scope, team roles defined and work began on the Receipt and Invoice Digitizer	02.02.2026
Prototype / First Draft	Basic Flask app with file upload, PaddleOCR extraction, and initial database storage working	10.02.2026
Mid-Term Review	Core features complete: OCR pipeline, Mistral extraction, user auth, dashboard UI; reviewed and refined	05.03.2026
Final Submission	Completed project with all features: chatbot, admin panel, dark mode,	11.03.2026

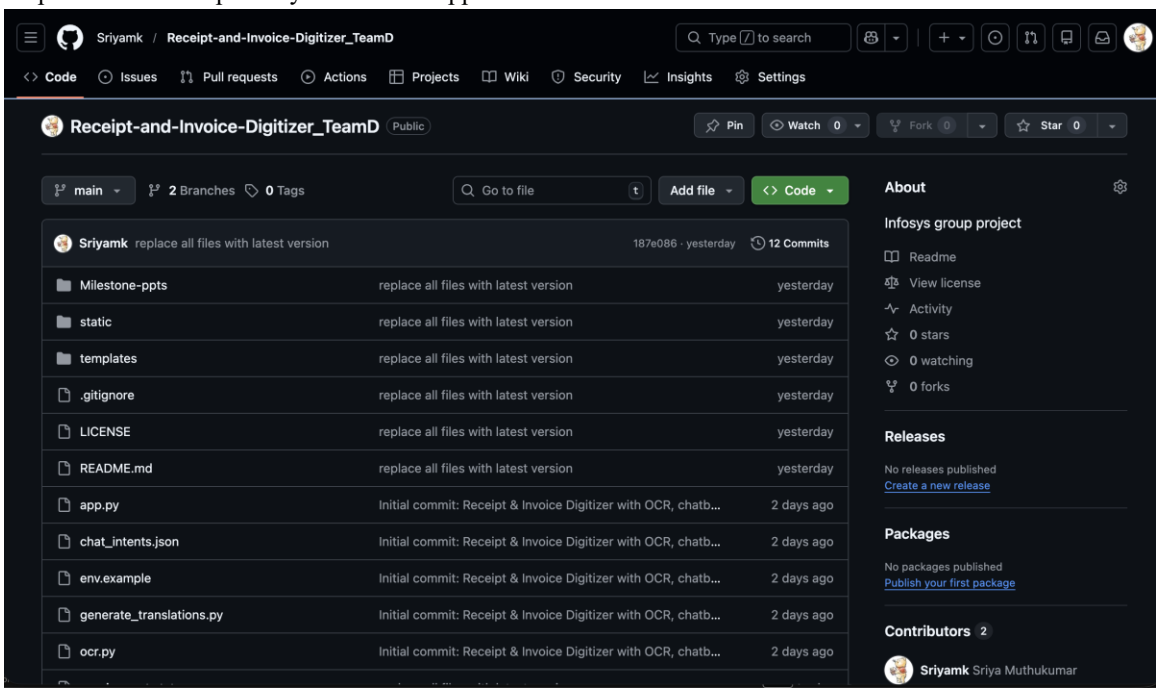
	multilingual, analytics	
Presentation	Prepared and delivered final project presentation showcasing features and outcomes	14.03.2026

5b. Project execution details

- Requirement Analysis – Gathered user needs around receipt digitization, defined project scope, and finalized objectives for OCR extraction, expense tracking, and AI-powered querying.
- Technology Selection – Chose PaddleOCR for text extraction, Mistral (via Ollama) as the local LLM, Flask for the backend, SQLite for storage, and vanilla JS with Chart.js for the frontend.
- System Design – Designed the SQLite schema for users and receipts, API endpoint structure, and UI wireframes with the pink/white brand theme.
- OCR Pipeline – Integrated PaddleOCR with custom result parsers to handle both PaddleOCR 2.x and 3.x output formats; added PyMuPDF for PDF-to-image conversion.
- LLM Extraction – Implemented Mistral-based structured extraction of Vendor, Invoice Number, Date, Total Amount, CGST, SGST, and GST; added regex fallbacks for robustness.
- Frontend Development – Built the landing page, login/registration pages, user dashboard, bill digitizer page, and admin dashboard with responsive design and dark mode support.
- Backend Development – Developed all Flask API routes for upload, receipts CRUD, analytics, chatbot, authentication, and admin; implemented bcrypt hashing and JWT-based Google OAuth.
- Chatbot Integration – Built a floating chat widget with Mistral intent detection, direct DB query routing for structured intents, and full Mistral fallback for general questions.
- Feature Enhancements – Added multilingual i18next support (6 languages), dark mode CSS variables, forgot/reset password via Flask-Mail, and an admin usage dashboard.
- Testing & Deployment – Conducted end-to-end testing of upload, OCR, extraction, storage, chatbot, and analytics; resolved merge conflicts; updated GitHub repository with final code.

6. Snapshots / Screenshots

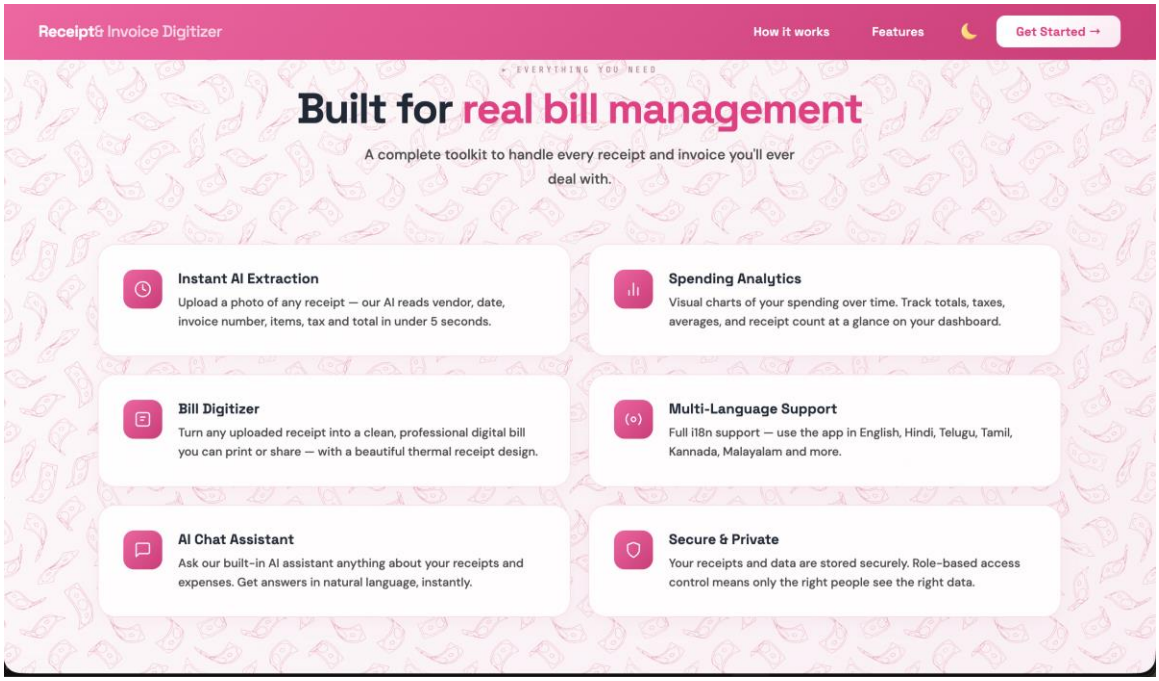
Step 1: Clone the repository and run the application.



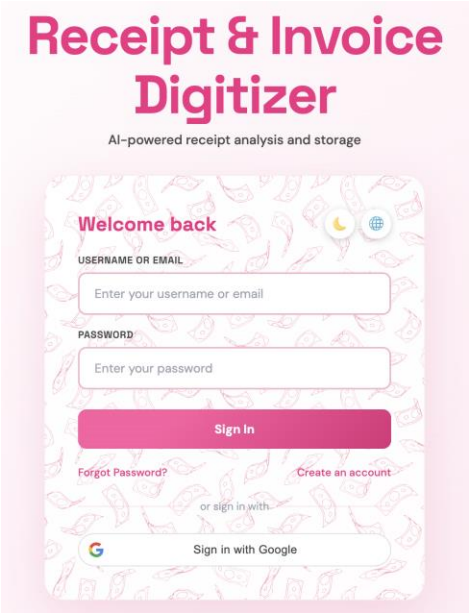
GitHub Repository: https://github.com/Sriyamk/Receipt-and-Invoice-Digitizer_TeamD

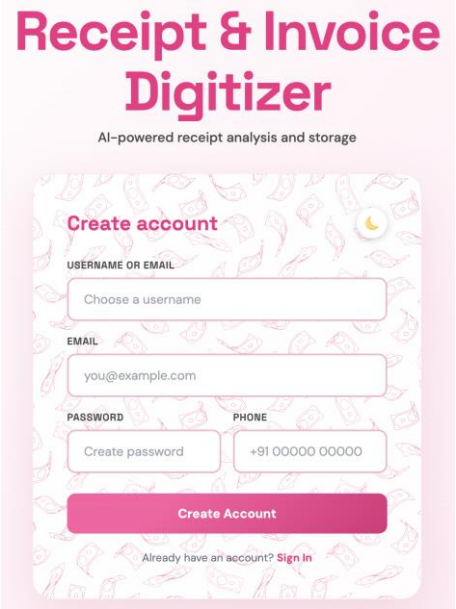
Run the application with: python app.py (after starting Ollama with ollama serve)

Step 2: Landing Page – Features the project introduction, navigation bar, and a 'Get Started' call-to-action.

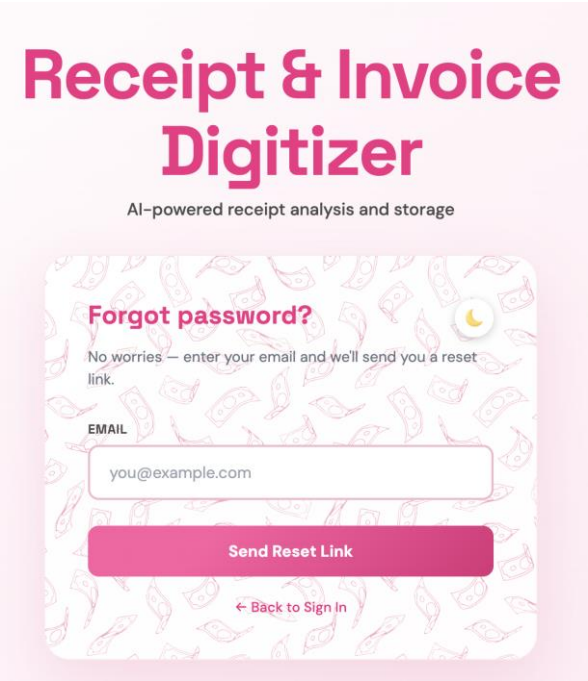


Step 3: Registration & Login Pages – Users register with a username, email, and password (bcrypt-hashed). Login supports both credentials and Google OAuth.

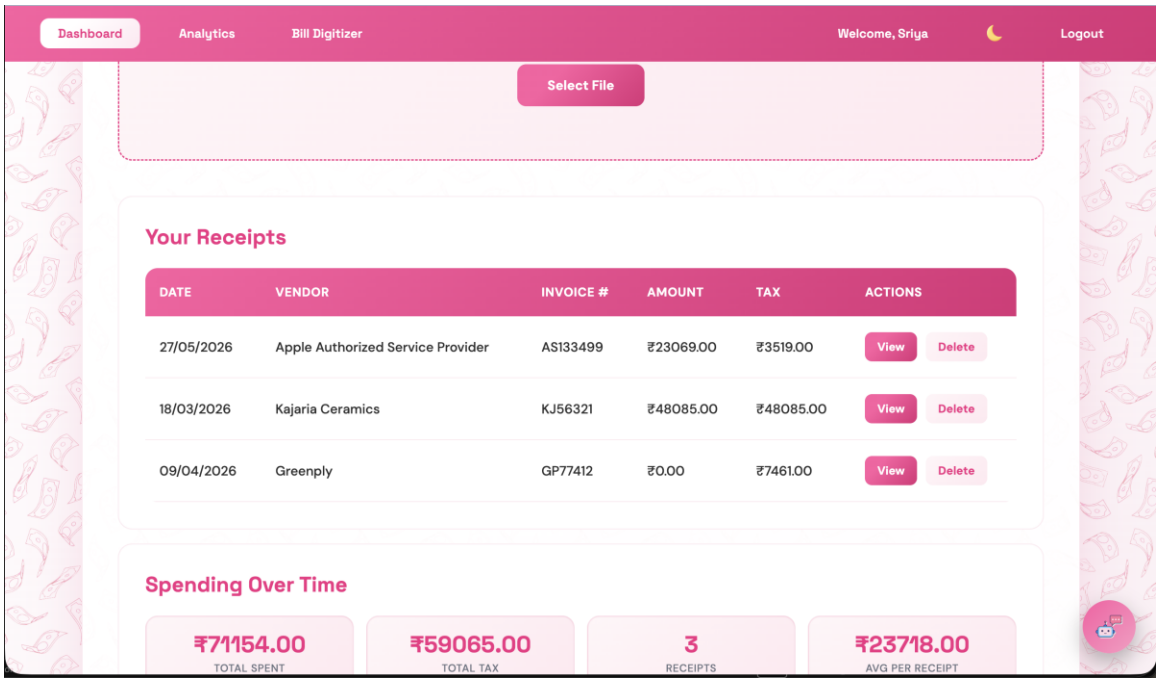




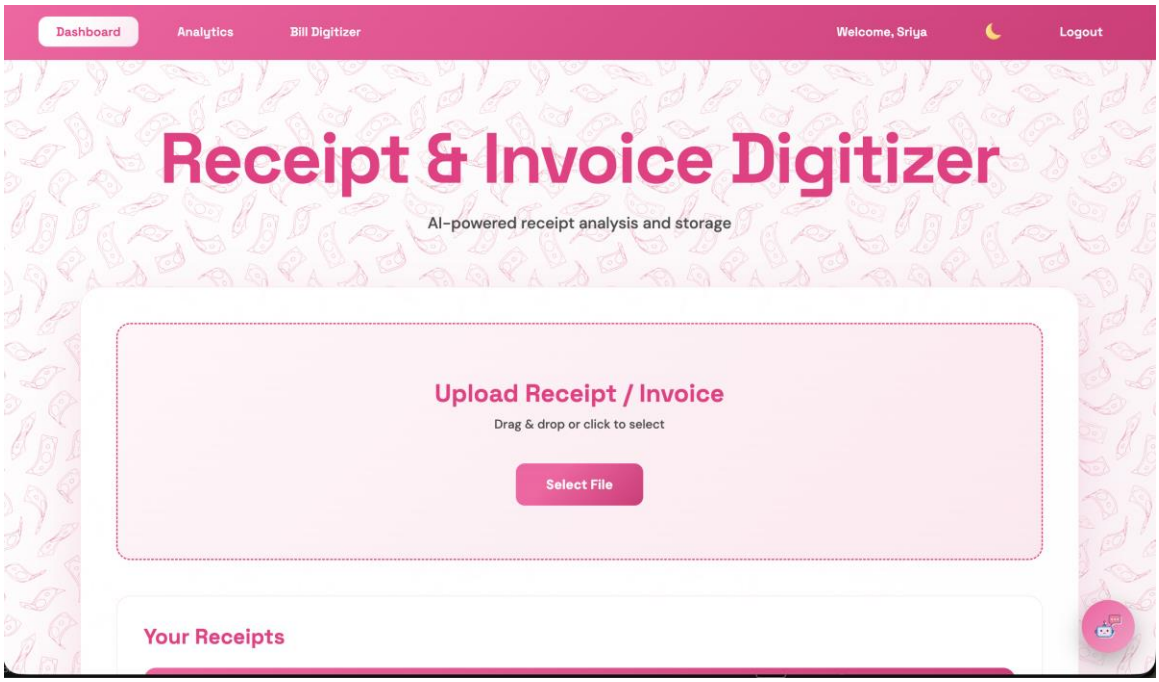
Step 4: Forgot Password / Reset Password – Users can request a password reset via email, and set a new password through a secure token-based link.



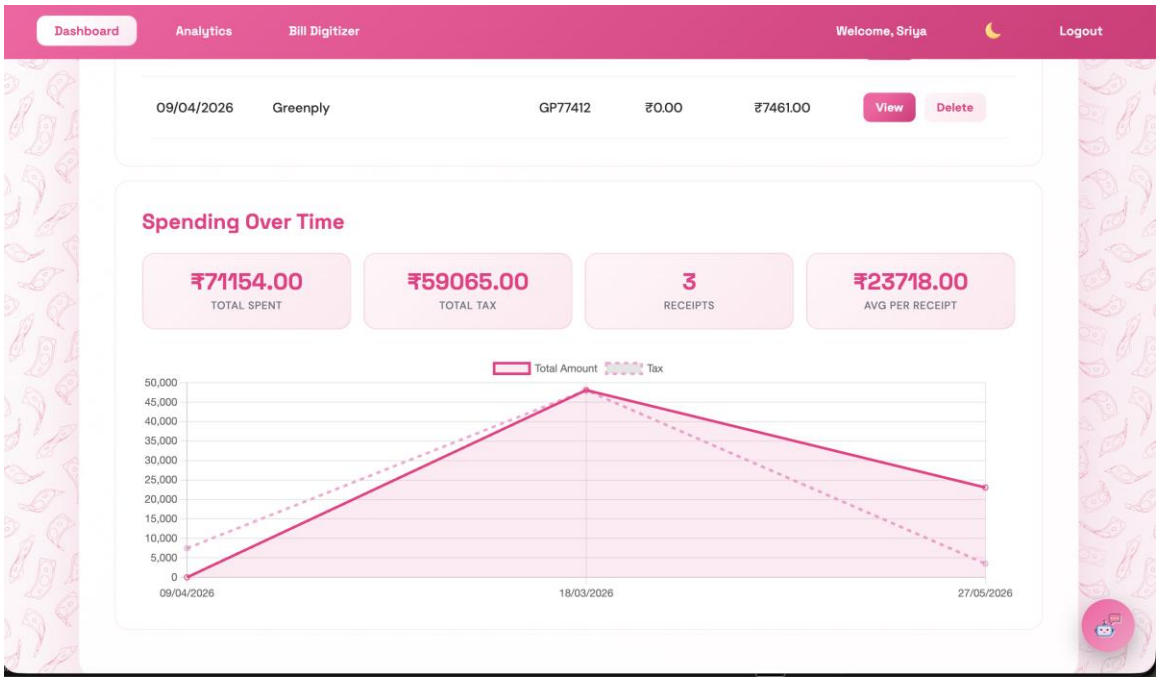
Step 5: User Dashboard – Displays all uploaded receipts in a table (Date, Vendor, Invoice #, Amount, Tax) with View and Delete actions.



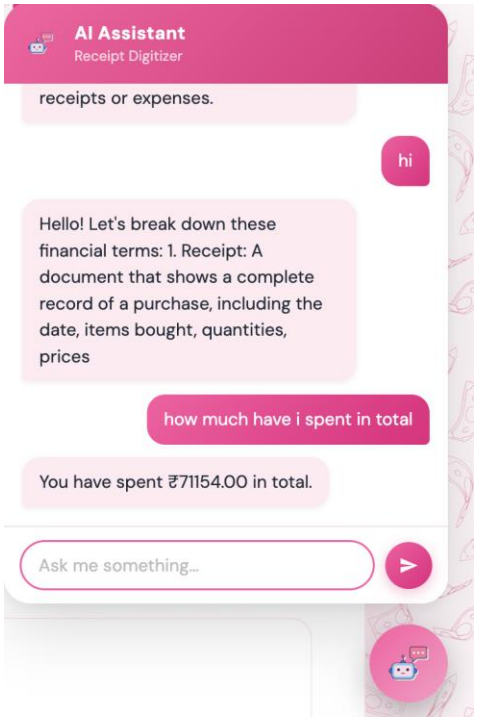
Step 6: Upload & OCR Extraction – Users drag-and-drop or select a receipt image/PDF. The system runs PaddleOCR, passes the text to Mistral, and saves extracted fields automatically.



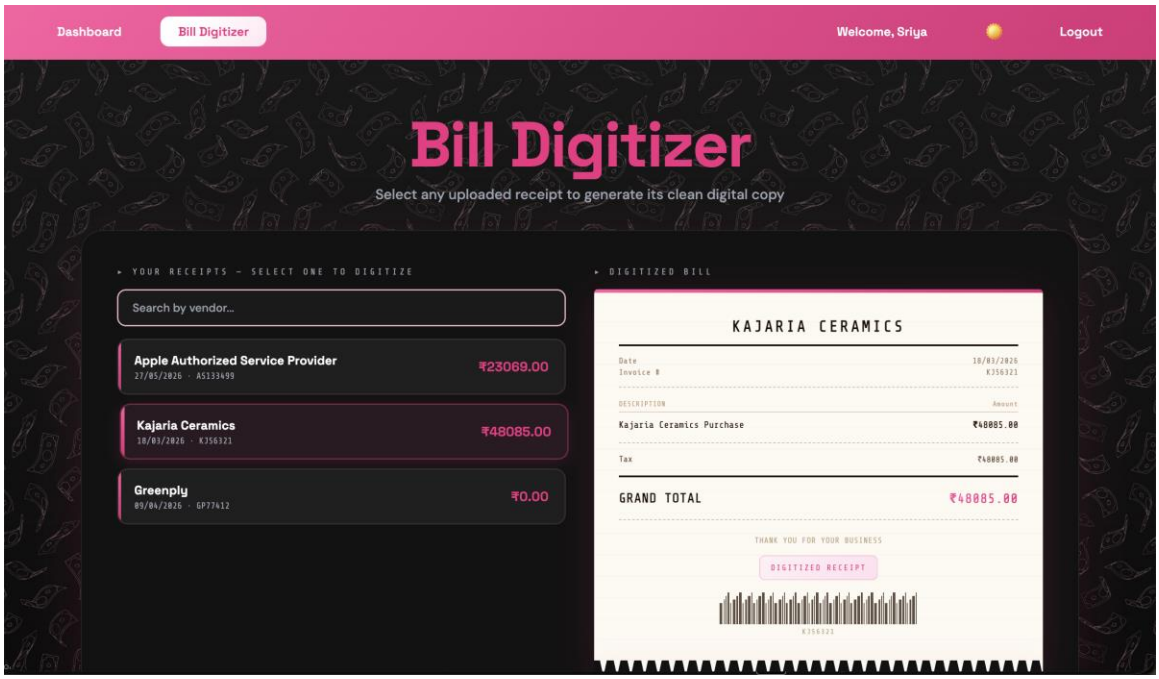
Step 7: Analytics Section – Chart.js line chart showing daily spending trends; four stat pills displaying Total Spent, Total Tax, Receipt Count, and Average per Receipt.



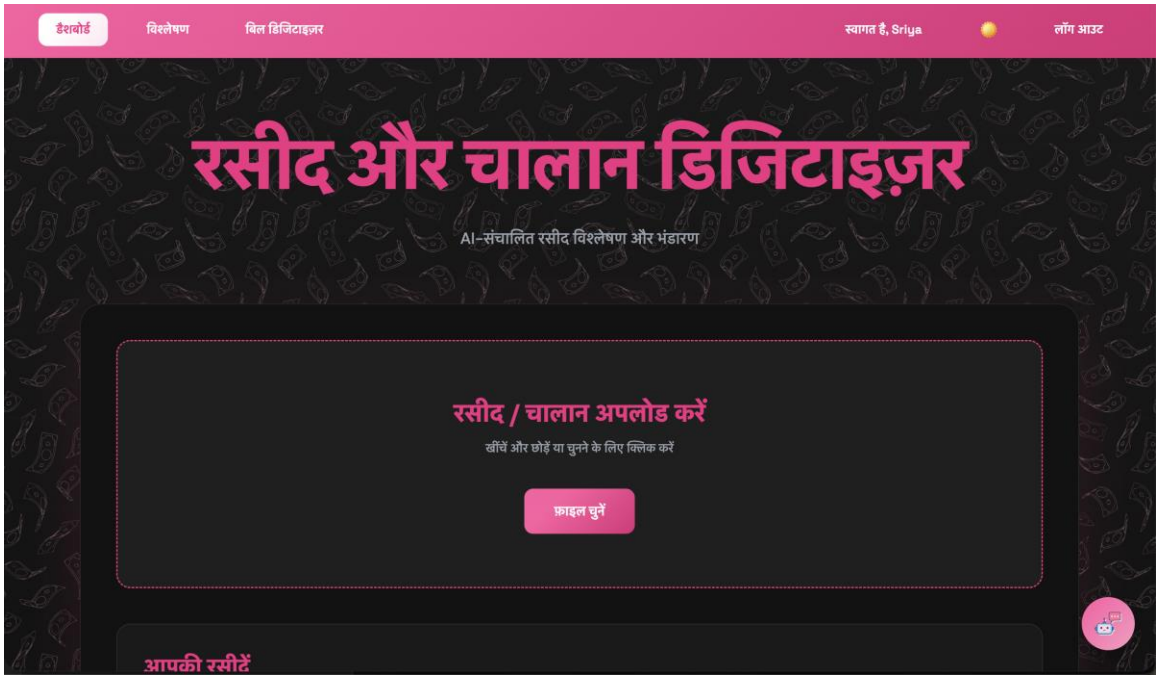
Step 8: AI Chatbot Widget – Floating chat button opens a chatbot powered by Mistral that answers natural language queries about the user's expenses in real time.



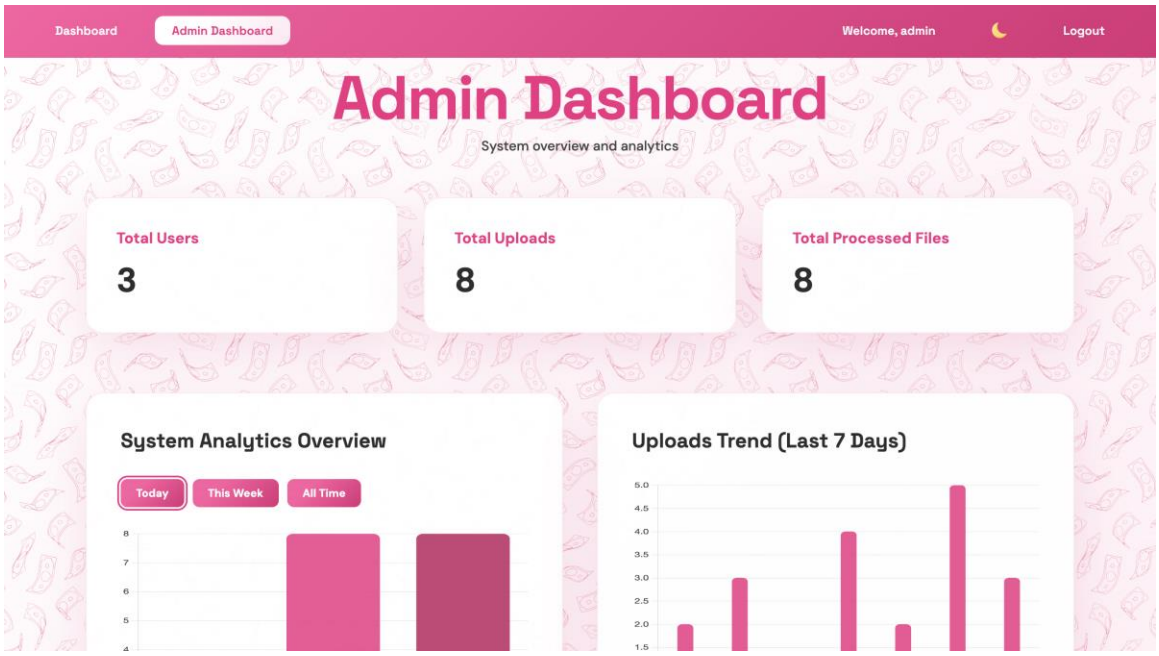
Step 9: Dark Mode – Full dark theme toggled via a moon/sun button in the navbar; persisted in localStorage across sessions.



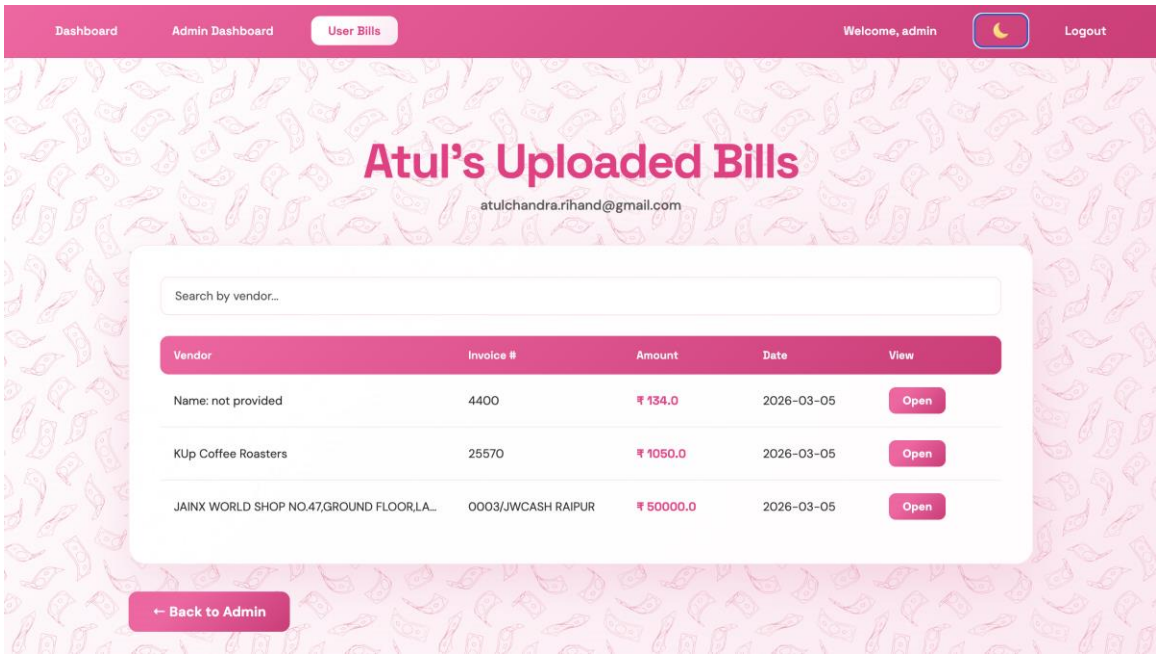
Step 10: Multilingual Support – Language selector switches the interface between English, Hindi, Telugu, Tamil, Kannada, and Malayalam using i18next.



Step 11: Admin Dashboard – Accessible to admin users only; shows total users, total uploads, uploads today, uploads this week, recent upload activity, and a per-user overview.



Step 12: Admin – User Receipt View – Admin can click any user to view a detailed list of all receipts uploaded by that user.



7. Challenges Faced

- PaddleOCR Version Differences – PaddleOCR 2.x and 3.x return results in different formats (nested list vs. dict with rec_texts), requiring a custom parser to handle both.

- Mistral Tax Hallucination – On split-line invoice formats where the tax value appears on the line before its label, Mistral would hallucinate incorrect values. Resolved by adding a regex-based primary tax extractor on the raw OCR text.
- Ollama Connection Errors – When Ollama was not running, unhandled ConnectionError exceptions crashed the Flask app. Fixed by wrapping all Ollama calls in try/except with graceful fallback responses.
- PDF Support – Converting PDFs to images for OCR required PyMuPDF (fitz), which is not always straightforward to install cross-platform. Documented clearly in requirements and setup.
- Git Merge Conflicts – Six team members working on parallel branches created frequent merge conflicts. Resolved through regular pull requests, code reviews, and conflict resolution sessions.
- i18n Welcome Username Bug – The i18next translation logic overwrote the Jinja-rendered username, showing 'User' instead of the actual username. Fixed by capturing the username from the DOM before any translation runs.
- Dark Mode Consistency – Ensuring all UI components (tables, modals, upload zones, chatbot, charts) respected dark mode CSS variables required careful auditing of every element.
- Session Security – Maintaining secure user sessions, preventing unauthorized access to admin routes, and protecting upload file endpoints required careful implementation of session checks across all routes.

8. Learnings & Skills Acquired

- OCR & Document Processing – Gained hands-on experience with PaddleOCR for text extraction from images and PDFs, including handling different output formats across library versions.
- LLM Integration – Learned to prompt and integrate a local LLM (Mistral via Ollama) for structured data extraction from unstructured OCR text, including handling hallucinations with fallback logic.
- Full-Stack Web Development – Built a complete Flask application with user authentication, file uploads, REST APIs, SQLite database, and a feature-rich frontend using HTML/CSS/JavaScript.
- Data Visualization – Implemented interactive Chart.js graphs for expense analytics, connected to real-time data from the Flask backend.
- Multilingual UI Development – Integrated i18next for runtime language switching across 6 languages, learning best practices for i18n in web applications.
- Secure Authentication – Implemented bcrypt password hashing, session management, JWT decoding for Google OAuth, and secure password reset flows with Flask-Mail.
- Collaborative Git Workflow – Practiced branching, pull requests, code reviews, and conflict resolution across a 6-member team over a 40-day period.
- Prompt Engineering – Learned to craft effective LLM prompts that return consistent, structured JSON output for invoice field extraction.
- CSS Variables & Theming – Implemented a full dark/light mode system using CSS custom properties, ensuring consistent theming across all UI components.
- Problem-Solving Under Constraints – Tackled real-world integration challenges (OCR version differences, LLM errors, i18n bugs) and developed practical debugging and fallback strategies.

9. Testimonials from team

Member 1:

"This project gave me a deep understanding of how AI tools like OCR and LLMs can be integrated into real-world applications. Building the frontend — from the landing page to the bill digitizer and receipt dashboard — taught me how to design user-friendly interfaces that are both functional and visually polished."

Member 2:

"Working on the database architecture and the admin dashboard gave me practical experience in designing scalable data models and building role-based access control. I now have a strong understanding of how backend systems power real applications."

Member 3:

"Researching OCR options, implementing the multilingual interface with i18next, and building the forgot password flow gave me exposure to areas I had never explored before. I gained confidence in both frontend development and feature research."

Member 4:

"Integrating PaddleOCR and Mistral into the backend was the most technically challenging and rewarding part of this project. I learned how to handle LLM hallucinations, build fallback logic, and create a robust AI-powered extraction pipeline."

Member 5:

"Building the login and password reset pages taught me how to implement secure authentication flows end-to-end. Working alongside the team and integrating my work with theirs strengthened my understanding of collaborative development."

Member 6:

"Handling documentation for this project helped me understand the full scope of what the team built. Writing clear technical reports improved my ability to communicate complex systems concisely and professionally."

10. Conclusion

The Receipt and Invoice Digitizer project successfully delivered a complete, AI-powered solution for automating receipt management and expense tracking. By combining PaddleOCR for text extraction, Mistral LLM for intelligent field parsing, Flask for the backend, and a polished JavaScript frontend, the team built a platform that is both technically sophisticated and practically useful.

The project went beyond the base requirements by adding significant feature enhancements: a real-time AI chatbot, spending analytics dashboard, dark mode, multilingual support across 6 languages, an admin panel, and a secure authentication system with Google OAuth and password reset. These additions reflect the team's initiative and commitment to delivering a production-quality application.

The 40-day internship provided invaluable hands-on experience in full-stack development, AI integration, collaborative Git workflows, and real-world problem solving — skills that will serve every team member well in their future careers.

11. Acknowledgements

We would like to express our sincere gratitude to Infosys Springboard for providing us the opportunity to work on the Receipt and Invoice Digitizer project and gain hands-on experience in AI, OCR, and full-stack web development.

We are especially thankful to **Miss. Shakti Gopalakrishnan,** our mentor for their constant guidance, support, and valuable insights throughout the internship. Their mentorship helped us navigate technical challenges and grow both as developers and as a team.

We would also like to thank each team member — Atul Chandra, Shaik Mastan, Thrishala Koduru, Sriya Muthukumar, Rithwik Pachipala, and Shailee Chouksey — for their dedication, collaboration, and hard work in bringing this project to life.

Finally, we appreciate everyone who contributed directly or indirectly to our learning and the successful completion of this project.